

Primera Corrida en el Cluster HPC UCR

CrossSim 2.0 — Verificación del entorno

Proyecto IE0499 I-2026

Autores:	Ricardo Chacon Carvajal (B11748) Jonathan D. Rodriguez Hernandez (B76490)
Supervisora:	M.Sc. Taina Ramirez Cortes
Fecha:	Junio 2026
Cluster:	HPC UCR — 172.16.24.2
Repositorio:	github.com/jodridez/ProyectoElectrico (rama: Ricardo)

Esta guía describe los pasos mínimos para verificar que CrossSim 2.0 corre correctamente en el cluster HPC de la UCR. El objetivo es ejecutar una sola corrida de training (1 run, 1 epoch) y una corrida de inferencia en condiciones ideales, confirmando que el entorno, la GPU y el pipeline funcionan antes de lanzar los experimentos completos del proyecto.

Que se hace en esta guía	Que NO se hace en esta guía
Instalar ambiente conda en el cluster Verificar GPU activa (A100) Training: 1 run, 1 epoch, material DWMTJ Inferencia ideal: condiciones sin error Confirmar resultados coherentes con WSL	Experimento completo (5 runs x 20 epochs) Integración de LUT de MoTe2 Inferencia con hardware no ideal Generación de LUTs desde datos del CELEQ

PASO 0 — Acceder al cluster

Requiere VPN activa si se accede desde fuera de la red UCR

- Conectar **GlobalProtect VPN-UCR**.
- Abrir **MobaXterm** → New Session → SSH.
- Host: **172.16.24.2** | Port: 22 | User: <user>.

IMPORTANTE: No correr calculos en el nodo de acceso. Solo instalar software, editar archivos y enviar jobs con sbatch.

PASO 1 — Verificar quota disponible

La quota por defecto es 25 GB — el repo pesa ~3 GB

```
quota -s
```

Nota: Si necesitas mas espacio: <https://soporteci.ucr.ac.cr/> Pedir ampliacion de quota con justificacion del proyecto.

PASO 2 — Instalar Miniforge3

Solo se hace una vez por cuenta de usuario

```
wget https://git.ucr.ac.cr/hpc/scripts-instalacion/-/raw/main/miniforge3_install.sh
bash miniforge3_install.sh
```

```
# Cerrar y reabrir la sesion SSH, luego verificar:
mamba --version
```

PASO 3 — Crear el ambiente conda

El cluster tiene CUDA 12.5 — usar cupy-cuda12x, no cupy-cuda13x

```
mamba create --name crosssim python=3.12
mamba activate crosssim
```

```
# setuptools compatible (requerido para pkg_resources de CrossSim)
pip install "setuptools<70"
```

```
# Stack cientifico
pip install "numpy<2.0" scipy pandas matplotlib pillow
```

```
# TensorFlow con soporte GPU
pip install "tensorflow[and-cuda]"
```

```
# CuPy para CUDA 12 (NO cuda13x)
pip install cupy-cuda12x
```

Nota: Si aparece ModuleNotFoundError: No module named 'pkg_resources' mas adelante, ejecutar: pip install "setuptools<70"

PASO 4 — Clonar el repositorio

Directamente desde GitHub dentro del cluster

```
cd ~
git clone https://github.com/jodridez/ProyectoElectrico.git
cd ProyectoElectrico
git checkout Ricardo
git submodule update --init --recursive
```

```
# Verificar estructura
ls
```

PASO 5 — Verificar GPU en sesion interactiva

Antes de enviar cualquier job, confirmar que la A100 es visible

```
# Abrir sesion interactiva en nodo GPU
srun -p gpu -G 1 --pty bash -l

# Activar ambiente
mamba activate crosssim

# Verificar TensorFlow ve la GPU
python3 -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"

# Verificar CuPy ve la GPU
python3 -c "import cupy; print(cupy.cuda.runtime.getDeviceCount())"

# Cerrar sesion interactiva
exit
```

Esperado: TensorFlow debe mostrar: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')] |
CuPy debe mostrar: 1

IMPORTANTE: Si la GPU no aparece, NO continuar. Verificar que cupy-cuda12x y tensorflow[and-cuda] se instalaron sin errores.

PASO 6 — Modificar MLP training.py para una corrida minima

Cambiar Nruns=1 y Nepochs=1 para el smoke test

Antes de enviar el job, editar **training/MLP_training.py** para reducir los parametros al minimo. Esto permite verificar que todo funciona sin esperar horas:

```
# Editar con nano o vim:
nano ~/ProyectoElectrico/training/MLP_training.py

# Cambiar estos valores:
Nruns   = 1    # una sola corrida
Nepochs = 1    # una sola epoca
batchsize = 200 # mas rapido que el default de 10
useGPU   = True # confirmar que esta activo
```

Nota: Para los experimentos definitivos del proyecto estos valores deben cambiarse a Nruns=5, Nepochs=20. Esta modificacion es solo para el smoke test.

PASO 7 — Crear y enviar el script de training

Job minimo para verificar que el pipeline de training funciona en el cluster

Crear el archivo **run_smoketest_training.sh** en /home/<user>/ProyectoElectrico/:

run_smoketest_training.sh

Cola: gpu | Tiempo: 2 horas | 1 run, 1 epoch, material DWMTJ

```
#!/bin/bash -l
#SBATCH --partition=gpu
#SBATCH --time=2:00:00
#SBATCH --nodes=1
#SBATCH --gpus-per-node=1
#SBATCH --ntasks=1
#SBATCH --job-name="crosssim_smoketest"
#SBATCH --output=smoketest_training_%j.out
#SBATCH --error=smoketest_training_%j.err
#SBATCH --mail-user=CORREO@ucr.ac.cr
#SBATCH --mail-type=BEGIN,END,FAIL

# Sustituir CORREO con el correo institucional @ucr.ac.cr

echo "====="
echo "Job ID : $SLURM_JOB_ID"
echo "Nodo   : $SLURMD_NODENAME"
echo "Inicio : $(date)"
echo "====="

mamba activate crosssim

# Verificar GPU
python3 -c "import tensorflow as tf; print('GPU:', tf.config.list_physical_devices('GPU'))"

cd /home/$USER/ProyectoElectrico/training

# Nruns=1, Nepochs=1, batchsize=200 configurados en MLP_training.py
python MLP_training.py

echo "====="
echo "Fin      : $(date)"
echo "====="
```

Enviar a cola y monitorear:

```
sbatch run_smoketest_training.sh

# Verificar que entro a cola
squeue --me

# Ver output en tiempo real
tail -f smoketest_training_<JOBID>.out

# Ver en que nodo esta corriendo
squeue --me
# Columna NODELIST muestra el nodo, ej: cngpu001

# Monitorear GPU desde ese nodo
ssh -t cngpu001 nvidia-smi
```

Esperado: Training exitoso: Numeric ~94-96%, Single LUT ~92%, Multi LUT ~90%. Tiempos menores que en WSL/RTX 4060 gracias a la A100.

PASO 8 — Convertir modelos y enviar job de inferencia

Convertir .npz a .h5 antes de correr inferencia

El training genera archivos **.npz**. La inferencia requiere **.h5**. La conversion se hace en una sesion interactiva rapida:

```
# Sesion interactiva para convertir (no requiere GPU)
srun -n1 --pty bash -l

mamba activate crosssim
cd ~/ProyectoElectrico/training

# Verificar que existen los .npz
ls cross_sim_models/*.npz

# Convertir
python npz_h5.py

# Verificar conversion
ls cross_sim_models/*.h5

exit
```

Nota: Si cross_sim_models/ esta vacio, el training no guardo los modelos. Verificar que saveCrossSimModels = True en MLP_training.py y repetir el Paso 7.

Crear el archivo **run_smoketest_inference.sh**:

run_smoketest_inference.sh

Cola: gpu | Tiempo: 2 horas | Condiciones ideales, alpha_error=0

```
#!/bin/bash -l
#SBATCH --partition=gpu
#SBATCH --time=2:00:00
#SBATCH --nodes=1
#SBATCH --gpus-per-node=1
#SBATCH --ntasks=1
#SBATCH --job-name="crosssim_inf_smoketest"
#SBATCH --output=smoketest_inference_%j.out
#SBATCH --error=smoketest_inference_%j.err
#SBATCH --mail-user=CORREO@ucr.ac.cr
#SBATCH --mail-type=BEGIN,END,FAIL

# Verificar antes de enviar que inference_config.py tenga:
#   alpha_error = 0.00
#   alpha_noise = 0.00
#   infinite_on_off_ratio = True

echo "====="
echo "Job ID : $SLURM_JOB_ID"
echo "Nodo   : $SLURMD_NODENAME"
echo "Inicio : $(date)"
echo "====="

mamba activate crosssim
cd /home/$USER/ProyectoElectrico/inference

python run_inference.py

echo "====="
echo "Fin      : $(date)"
echo "====="

sbatch run_smoketest_inference.sh
squeue --me
tail -f smoketest_inference_<JOBID>.out
```

Esperado: Inferencia exitosa: precision ~80% en condiciones ideales. Si este numero coincide con el obtenido en WSL, el entorno del cluster esta correctamente configurado.

PASO 9 — Verificar resultados y reportar eficiencia

Confirmar que los numeros coinciden con los obtenidos en WSL

Una vez completados ambos jobs, verificar que los resultados son coherentes con los obtenidos previamente en WSL/RTX 4060:

Escenario	Resultado esperado	Referencia WSL
Training — Numeric (ideal)	~94-96%	94.96% (1 epoch, DWMTJ)
Training — Single LUT	~92%	92.41%
Training — Multi LUT	~90%	90.91%
Inferencia ideal	~80%	~80% (CrossSim 2.0 nativo)

Revisar eficiencia del uso de recursos:

```
# Reporte de eficiencia de todos mis jobs
reportseff -u $USER

# Ver detalles de un job terminado
sacct -j <JOBID>

# Bajar los archivos de output al equipo local
# (desde MobaXterm o PowerShell en Windows)
scp <user>@172.16.24.2:/home/<user>/ProyectoElectrico/smoketest_*.out .
scp <user>@172.16.24.2:/home/<user>/ProyectoElectrico/smoketest_*.err .
```

Que sigue si el smoke test es exitoso

Con el entorno verificado, los siguientes pasos del proyecto son:

Paso siguiente	Accion
Experimento completo de training	Cambiar Nruns=5, Nepochs=20 en MLP_training.py sbatch run_training.sh
Integracion de LUT de MoTe2	Subir set_MoTe2.csv y reset_MoTe2.csv del CELEQ sbatch run_lut_gen.sh
Training con hardware MoTe2	Agregar bloque MoTe2 en MLP_training.py sbatch run_training.sh
Inferencia con hardware no ideal	Configurar Rmin/Rmax en inference_config.py sbatch run_inference_nonideal.sh

Nota: Ver Guia_Cluster_HPC_CrossSim.pdf para el flujo completo de experimentos con MoTe2.